Project Proposal & Scope Document

DeployGuard:A Reliable Repository Scanning and Deployment Orchestration System

Submitted By

Hassan Sajjad -232460

Faria Fatima - 233169

Tania Khawar -232378

Supervisor

Sir Yaseen Mushtaq

Co-Supervisor

Sir Asim Ali Fayyaz



Department of Computer Science

Air University Islamabad (AU)

Supervisor Meeting Log

Date	Supervisor	Discussion Points	Supervisor Signature
05-02-2026	Sir Yaseen Mushtaq	Initial project concept for "DeployGuard" presented. Discussion focused on solving the DevOps skill gap for SMEs via a "zero-configuration" pipeline. Feasibility of mapping code to infrastructure automatically and using AI for troubleshooting was reviewed.	
11-02-2026	Sir Yaseen Mushtaq	Technology stack finalized: Next.js for frontend, NestJS/Node.js for the orchestration engine, and PostgreSQL for audit logging. Supervisor approved the use of HashiCorp Terraform for IaC and Redis/BullMQ for managing asynchronous deployment queues.	
18-02-2026	Sir Yaseen Mushtaq	Detailed review of the Rule-Based Stack Detection engine to eliminate AI hallucination risks. finalized "Shift-Left" security requirements using Aqua Trivy and pre-deployment financial transparency via Infracost API integration.	
26-02-2026	Sir Yaseen Mushtaq	Final module list confirmed, including AWS S3/DynamoDB state locking, AI-powered log analysis, and the "No Lock-in" Terraform export feature. Project proposal signed.	

Table of Contents

1. Introduction 4
2. Related System Analysis / Literature Review 4
3. Problem Statement 4
4. Problem Solution / Objectives of the Proposed System 5
   4.1 Objectives 5
5. Scope 5
6. Modules 6
   6.1. User Authentication & Audit Logging 6
   6.2. Project & Workspace Management 6
   6.3. Rule-Based Stack Detection Engine 6
   6.4. DevOps Template Engine & Pre-Flight Validation 7
   6.5. CI/CD Pipeline & Asynchronous Queuing (Redis) 7
   6.6. Security Scanning and Policy Enforcement System 7
   6.7. Predictive Cost Analysis (FinOps) 8
   6.8. Infrastructure Provisioning & Service Discovery 8
   6.9. Distributed State Management 9
   6.10. Persistent Storage & Backup Recovery (AWS EFS) 9
   6.11. Zero-Downtime Container Orchestration & Event-Driven Rollbacks 9
   6.12. Dual-Stack Monitoring & Observability 10
   6.13. AI-Powered Log Analysis & Chatbot 10
   6.14. Alerting & Notification System 11
   6.15. Infrastructure Lifecycle & Demolition (Destroy) 11
   6.16. Vendor Independence & Terraform Export (No Lock-in) 11
   6.17. Subscription & Billing Management 11
7. System Limitations/Constraints 12
8. Data Gathering Approach 12
9. Project Stakeholders and Roles 12
10. Module based Work Division 13
11. Gantt Chart 14
12. Mockups 14
   12.1. Home/Dashboard Page 15
   12.2. Project Creation/Work Space Page 15
   12.3. Repository Stack Detection & Pre-Flight Page 15
   12.4. Chatbot Page 16
13. Conclusion 16
14. References 17

1. Introduction

The software development sector is growing rapidly, yet startups and Small-to-Medium Enterprises (SMEs) remain heavily constrained by the complexities of cloud deployment. Developers often struggle to configure reliable infrastructure, while businesses are deprived of the cost savings associated with optimized cloud primitives due to a severe DevOps skill gap. Additionally, traditional manual deployments are highly prone to human error, resulting in security vulnerabilities, unstandardized environments, and slow-release cycles.

Recognizing these challenges, we propose the development of "DeployGuard," an enterprise-ready, cloud-native DevOps orchestration platform. This platform transforms the deployment experience by introducing a "zero-configuration" pipeline that maps code to infrastructure automatically via safe, rule-based scanning. Furthermore, an integrated Conversational AI Assistant actively troubleshoots deployment errors, bridging the technical gap for developers. DeployGuard provides a strictly governed, secure pipeline that enhances deployment efficiency and operational confidence.

2. Related System Analysis / Literature Review

Application Name	**Weakness**	**Proposed Project Solution (DeployGuard)**
Raw AWS Console	Extremely steep learning curve; requires deep DevOps knowledge to configure VPCs ALBs, and ECS manually.	Abstracts the complexity of AWS, using an automated orchestration engine to deploy code seamlessly without requiring raw console access.
Standard PaaS (e.g., Heroku)	Heavily abstracted but very expensive at scale. Lacks transparent cost estimation before deploying.	Deploys directly to the user's AWS account using Fargate Spot instances for massive cost savings, integrating Infracost for pre-deployment financial transparency.
Basic GitHub Actions	Only handles CI/CD; lacks built-in infrastructure provisioning (IaC) state locking and mandatory security gates.	Enforces mandatory Aqua Trivy security scanning and manages HashiCorp Terraform state locking via S3/DynamoDB to prevent concurrent deployment collisions.

3. Problem Statement

The absence of accessible, secure, and cost-effective cloud orchestration for SMEs represents

a significant bottleneck in software delivery. Existing solutions operate predominantly on two extremes: either heavily abstracted but expensive managed platforms, or raw cloud primitives that require dedicated, highly paid DevOps engineers. As a result, small teams rely on manual, unstandardized deployment scripts. Without integrated pre-deployment cost estimators or automated security gates, these teams frequently face unexpected cloud "bill shock" and inadvertently push vulnerable container images to production environments. Furthermore, when complex cloud deployments fail, developers waste hours parsing cryptic log files without clear-steps.

4. Problem Solution / Objectives of the Proposed System

To address the limitations of current workflows, DeployGuard introduces a robust orchestration platform. To completely eliminate the risks of AI code hallucination during containerization, the system utilizes strict rule-based repository scanning to map application stacks directly to highly optimized, pre-stored Docker templates. This automates architectural decisions while guaranteeing stability. Secure infrastructure provisioning via HashiCorp Terraform (with distributed state locking) ensures deployment reliability. Finally, to eliminate the friction of debugging, an integrated AI Assistant parses failed logs and provides plain-English solutions.

4.1 Objectives

- To develop a "zero-configuration" cloud deployment platform tailored for startups and SMEs.
- To eliminate manual configuration errors by utilizing rule-based repository scanning mapped to pre-stored DevOps Docker templates.
- To enforce mandatory "shift-left" security by integrating Aqua Trivy to block the deployment of critical vulnerabilities.
- To provide absolute financial transparency by integrating Infracost for pre-deployment AWS cost estimations.
- To prevent infrastructure state corruption during concurrent deployments using Amazon S3 and DynamoDB remote state locking.
- To integrate an AI-powered Troubleshooting Chatbot (LLM) that automatically analyzes failed deployment logs, pinpoints the exact point of failure, and provides actionable remediation steps.

5. Scope

The scope of this project is to develop a DevOps orchestration web application for seamless cloud deployments. Users will link their GitHub repositories, and the system will automatically scan, containerize, and deploy the application to AWS ECS Fargate Spot instances. The platform features robust CI/CD automation, a vulnerability reporting dashboard, pre-deployment cost analysis, and IaC provisioning using Terraform. The scope includes an integrated LLM chatbot strictly focused on platform guidance, log analysis, and error detection

to assist users when builds or deployments fail.

6. Modules

6.1. User Authentication & Audit Logging

- Allow users to securely sign up and log in using GitHub OAuth and standard email/password credentials.
- Implement role-based access control (RBAC) to differentiate between Admin, Developer, and Read-Only users.
- **Audit Logging:** Immutably record every system action (deployments, Trivy failures, cost approvals) in a highly available PostgreSQL database via TypeORM for enterprise compliance.

6.2. Project & Workspace Management

- Allow users to create isolated "Projects" within their workspace and link specific GitHub repositories to them.
- Enable users to select target branches for deployment and securely manage environment variables (.env).

6.3. Rule-Based Stack Detection Engine

Purpose: Autonomously identify technology stack and derive deployment requirements from repository structure.

Technical Implementation:

The detection engine performs shallow repository cloning to minimize network transfer while extracting configuration files needed for analysis. The system scans for ecosystem-defining files: package.json indicates Node.js, requirements.txt or pyproject.toml indicates Python, Gemfile indicates Ruby.

Ecosystem Identification: Once the primary language is detected, dependency manifests are parsed to identify framework signatures. For Node.js projects, the presence of "next" in package.json dependencies combined with next.config.js file confirms Next.js. Similarly, "express" identifies Express.js, "django" identifies Django, and "fastapi" identifies FastAPI.

Configuration Extraction: Framework-specific configuration files are analyzed to extract deployment parameters. For Next.js, the system parses next.config.js for port configuration, examines package.json scripts for build commands, and detects static export versus server-side rendering mode. For Django, settings.py is parsed to identify database requirements, static file paths, and WSGI server configuration.

Deployment Profile Generation: Detection results are structured into a deployment profile containing runtime version, framework type, package manager, build requirements, expected port, health check endpoint, database needs, and persistent storage requirements. This profile

drives all downstream template selection and infrastructure provisioning decisions.

Template Matching Logic: The system applies hierarchical matching to select deployment templates. Framework-specific templates are prioritized when available, such as selecting a Next.js SSR template for server-rendered applications or a Django WSGI template for Python web services. When frameworks are unrecognized but the language ecosystem is known, generic language-level templates provide fallback options. If no safe automatic match exists, the system requires user-provided custom Dockerfiles to maintain deployment correctness.
Deterministic Behavior: Identical repositories always produce identical detection results and template selections, ensuring reproducible builds without randomness or probabilistic inference. This eliminates hallucination risk inherent in AI-based approaches

6.4. DevOps Template Engine & Pre-Flight Validation

- Maintain a secure registry of optimized Docker templates utilizing distroless/alpine base images and multi-stage builds.
- **Pre-Flight Validation:** Generate a structured JSON report detailing the detected stack and injected template for user review before building.

6.5. CI/CD Pipeline & Asynchronous Queuing (Redis)

- **Asynchronous Execution:** Utilize Redis and BullMQ to queue long-running Terraform and Docker build jobs, ensuring the NestJS backend remains highly responsive.
- Automatically trigger GitHub Actions workflows, build the image, and immutably tag it with its unique Git commit hash.
- Push artifacts to AWS ECR and enforce **ECR Lifecycle Policies** to automatically purge untagged images older than 30 days to save cloud costs.

6.6. Security Scanning and Policy Enforcement System

Purpose: Enforce mandatory vulnerability analysis as a pre-deployment security gate.

Technical Implementation:

Aqua Trivy security scanner is invoked to analyze built container images for vulnerabilities in operating system packages and application dependencies. The scanner compares installed package versions against public CVE databases and returns structured findings.

Vulnerability Data Processing: The system parses Trivy's JSON output to extract vulnerability records containing CVE identifiers, severity classifications (CRITICAL, HIGH, MEDIUM, LOW), affected packages, installed versions, and fixed versions when available. Findings are normalized into database records for querying and historical analysis.

Policy Evaluation Engine: Scan results are evaluated through rule-based policy logic that

converts vulnerability data into deployment decisions. The policy engine implements security expertise as decision rules rather than relying on machine learning inference.

Critical and High severity findings trigger automatic deployment blocking because the platform enforces a zero-trust pre-deployment model. Medium severity findings above configurable thresholds require manual user review and approval. Low severity findings are recorded for awareness but do not block deployment. This evaluation converts raw scan data into governed deployment decisions.

Remediation Guidance Generation: When deployments are blocked, the system analyzes fixed version fields from vulnerability records and generates specific upgrade commands mapped to the detected package manager syntax, such as "npm update package@version" or "pip install package==version". Base image vulnerabilities trigger recommendations for updated image tags.

Audit Trail Integration: All scan results, policy decisions, blocking actions, and user overrides are persisted in immutable PostgreSQL audit logs with timestamps and user context, supporting compliance requirements and security posture tracking over time.

6.7. Predictive Cost Analysis (FinOps)

Purpose: Provide pre-deployment infrastructure cost transparency and prevent budget overruns.

Technical Implementation:

Before provisioning any infrastructure, the system generates a Terraform execution plan describing all resources to be created, modified, or destroyed. This plan is parsed and submitted to the

Infracost API, which returns real-time AWS pricing data for each planned resource.

Cost Analysis Processing: The raw pricing response is transformed into user-facing cost breakdowns organized by resource type including compute charges for ECS Fargate, load balancer costs, database instance costs, storage costs, and data transfer costs. Estimated monthly run-rate is calculated and presented alongside resource-level detail.

Approval Workflow Integration: Cost estimates are evaluated against subscription tier limits and user-configured warning thresholds. If estimated costs exceed tier limits, deployment is blocked and upgrade prompts are displayed. If costs exceed warning thresholds, explicit user approval is required before infrastructure provisioning begins. This converts cost estimation into a governed financial checkpoint within the deployment pipeline.

6.8. Infrastructure Provisioning & Service Discovery

- Execute HashiCorp Terraform to construct the AWS networking topology (VPCs, public/private-subnets,NATGateways).

- **Multi-Service Support:** Provision AWS Cloud Map for internal Service Discovery, allowing interconnected microservices to communicate securely over local DNS without hitting the public internet.

6.9. Distributed State Management

Purpose: Prevent infrastructure state corruption during concurrent deployment operations.

Technical Implementation:

Terraform state files are stored remotely in AWS S3 with versioning enabled, ensuring every infrastructure modification creates a recoverable state version. State file paths are organized hierarchically by user and project to enforce complete isolation between deployments.

Distributed Locking Protocol: Before executing infrastructure-changing operations, the system acquires a distributed lock through DynamoDB conditional writes. Lock records contain project identifier, deployment identifier, user context, timestamp, and heartbeat interval. If the conditional write succeeds, the lock is acquired and Terraform execution proceeds. If it fails due to existing lock, the deployment enters a project-specific queue.

Heartbeat and Liveness Detection: While holding a lock, a background process updates the heartbeat timestamp every 30 seconds, proving the deployment process remains active. A separate monitoring process continuously inspects active locks and compares heartbeat timestamps against current time.

Orphaned Lock Recovery: When a lock's heartbeat has not updated for 5 minutes, the system marks it as orphaned, indicating process crash or network partition. Safety checks verify that no active Terraform process owns the lock before force-releasing it through deletion. The orphaned deployment is marked as failed, and the next queued deployment for that project is automatically processed.

State Corruption Detection: Automated validation checks JSON schema correctness, checksum integrity, resource count consistency, and dependency graph validity. If corruption is detected, the system retrieves the previous state version from S3 and prompts for recovery decision.

This queue orchestration and locking coordination represents custom distributed systems engineering ensuring deployment reliability under concurrent load.

6.10. Persistent Storage & Backup Recovery (AWS EFS)

- Dynamically provision Amazon Elastic File System (EFS) for stateful workloads and persistent databases.
- Ensure data survival across container restarts by mounting EFS volumes with strict POSIX permissions, encrypted at rest via AWS KMS.

6.11. Zero-Downtime Container Orchestration & Event-Driven Rollbacks

- Deploy containers to serverless AWS ECS Fargate Spot instances with target-tracking CPU auto-scaling.
- **Spot Resilience:** Actively listen for AWS EventBridge 2-minute Spot Interruption notices to spin up replacement tasks instantly.
- **Layer 7 Health Checks:** Configure the ALB to constantly ping the /health endpoint. If a container fails, the system orchestrates a **Zero-Downtime Rollback** to the previously stable Git commit hash.

6.12. Dual-Stack Monitoring & Observability

- **Pipeline Stack:** Monitor GitHub Actions build times and Trivy scan durations.
- **Infrastructure Stack:** Stream (stdout) and (stderr) logs from ECS tasks to the frontend via Server-Sent Events (SSE). Integrate Prometheus to plot runtime telemetry (CPU/RAM usage, HTTP latency) on the dashboard.

6.13. AI-Powered Log Analysis & Chatbot

Purpose: Transform complex deployment failures into actionable remediation guidance through intelligent log analysis.

Technical Implementation:

The platform integrates pre-trained Large Language Model APIs for troubleshooting support.

Multi-Source Log Aggregation: When deployments fail, logs are collected from all execution stages including GitHub Actions build output, Terraform execution logs, CloudWatch container logs, and Trivy security scan results. These sources are correlated by deployment identifier so the system understands failure context.

Intelligent Preprocessing Pipeline: Raw logs typically contain 10,000+ lines of verbose output unsuitable for direct AI processing. The system implements custom error extraction using pattern matching to identify error keywords, stack traces, abnormal exit codes, and failure indicators. Routine noise such as successful operations and progress indicators is filtered out. The result is a condensed diagnostic context preserving technical meaning while fitting within model token limits, typically reducing to 50-200 essential lines.

Structured Prompt Engineering: Rather than sending raw logs to the AI, the system constructs structured prompts ensuring reliable output format. Prompts include role definition specifying DevOps expertise domain, deployment context with detected technology stack and failed stage, preprocessed error data from the extraction pipeline, output format instructions requiring root cause analysis and numbered remediation steps, and framework-specific documentation snippets retrieved from curated knowledge base to prevent hallucination.

Temperature and Parameter Tuning: API temperature is set to 0.3 for factual, deterministic responses rather than creative variations. Max token limits are configured to 1000 for detailed yet-concise-analysis.

Response Validation and Quality Control: AI responses are validated for required structure including root cause section, technical details section, and numbered remediation steps. Malformed responses trigger retry with clarified prompts or automatic fallback to alternative model. If both primary and fallback APIs fail, the system gracefully degrades to preprocessed logs without AI analysis, ensuring users always receive diagnostic information.

Conversational Context Management: The platform maintains short conversation history stored in PostgreSQL, allowing users to ask follow-up questions such as how to fix specific packages or why particular health checks failed. Context is pruned when exceeding five message pairs to prevent token overflow while preserving troubleshooting continuity.

Hallucination Prevention: Prompts explicitly instruct the model to base recommendations only on provided log evidence. AI responses include cross-references linking to actual log lines so users can verify suggestions against raw data in parallel log viewer.

This subsystem should be understood as prompt-engineered troubleshooting assistance built on custom log intelligence, not as trained AI. The engineering value lies in the preprocessing pipeline, deployment-aware prompt construction, response validation, and conversational debugging flow.

6.14. Alerting & Notification System

- Track the lifecycle of every deployment (Queued, Building, Scanning, Provisioning, Deployed, Failed).
- Integrate **AWS Simple Notification Service (SNS)** to dispatch real-time webhooks to email for critical events.

6.15. Infrastructure Lifecycle & Demolition (Destroy)

- Provide a dual-confirmation workflow that executes terraform destroy to safely tear down active AWS environments.
- Explicitly skip the destruction of persistent EFS volumes to prevent accidental data loss.

6.16. Vendor Independence & Terraform Export (No Lock-in)

- Prevent platform lock-in by providing a "One-Click Export" feature.
- Allow users to download a .zip archive containing their generated Terraform scripts,giving them 100% ownership of their infrastructure as code.

6.17. Subscription & Billing Management

- **Tiered Access:** Implementation of "Free" and "Pro" plans to manage resource limits (e.g., number of active projects).
- **Payment Integration:** High-level integration with a standard payment gateway for secure checkout.

Billing Dashboard: A simple interface for users to view their current plan, update

payment methods, and download past invoices.

- **Usage Tracking:** Provide alerts if a user reaches their tier limit.

7. System Limitations/Constraints

- **Cloud Provider Lock-in:** The system's highly optimized Terraform modules are tailored specifically to AWS APIs. Multi-cloud deployment (Azure/GCP) is out of scope.
- **Spot Instance Volatility:** Because the platform relies on AWS Fargate Spot for extreme cost savings, compute nodes may occasionally be reclaimed by AWS during periods of regional capacity shortages.
- **Repository Platform:** The CI/CD pipeline currently requires the source code to be hosted on GitHub.
- **Template Mapping Boundaries:** The system will strictly deploy frameworks that match its pre-stored DevOps templates. Highly unconventional stacks will require manual Dockerfile inclusion by the user.
- **Stack Deployment:** We are currently deploying javascript and python based web application

8. Data Gathering Approach

Data gathering for this complex orchestration system relies on a comprehensive study of cloud engineering documentation. We will dedicate substantial effort to comprehending AWS architectural best practices (ECS, Fargate, VPCs), HashiCorp Terraform documentation, and modern DevSecOps standards. Furthermore, we will continuously seek guidance from our supervisor, Sir Yaseen Mushtaq, to validate our infrastructure decisions and LLM prompt engineering for log analysisTools and Technologies

Tool/Technology	**Rationale**
Next.js (React)	Frontend Development, UI/UX, & Chatbot Interface
NestJS & Node.js	Backend Orchestration Engine
PostgreSQL	Relational Database for User/Project Data
Redis (BullMQ)	Asynchronous Task Queuing for Deployments
AWS (ECS, ECR, S3, ALB)	Cloud Infrastructure Provider
HashiCorp Terraform	Infrastructure as Code (IaC) engine
Aqua Trivy & Infracost	Security Scanning & FinOps Cost Estimation
GitHub Actions	CI/CD Pipeline Automation
LLM API	AI-Powered Troubleshooting and Log Parsing

9. Project Stakeholders and Roles

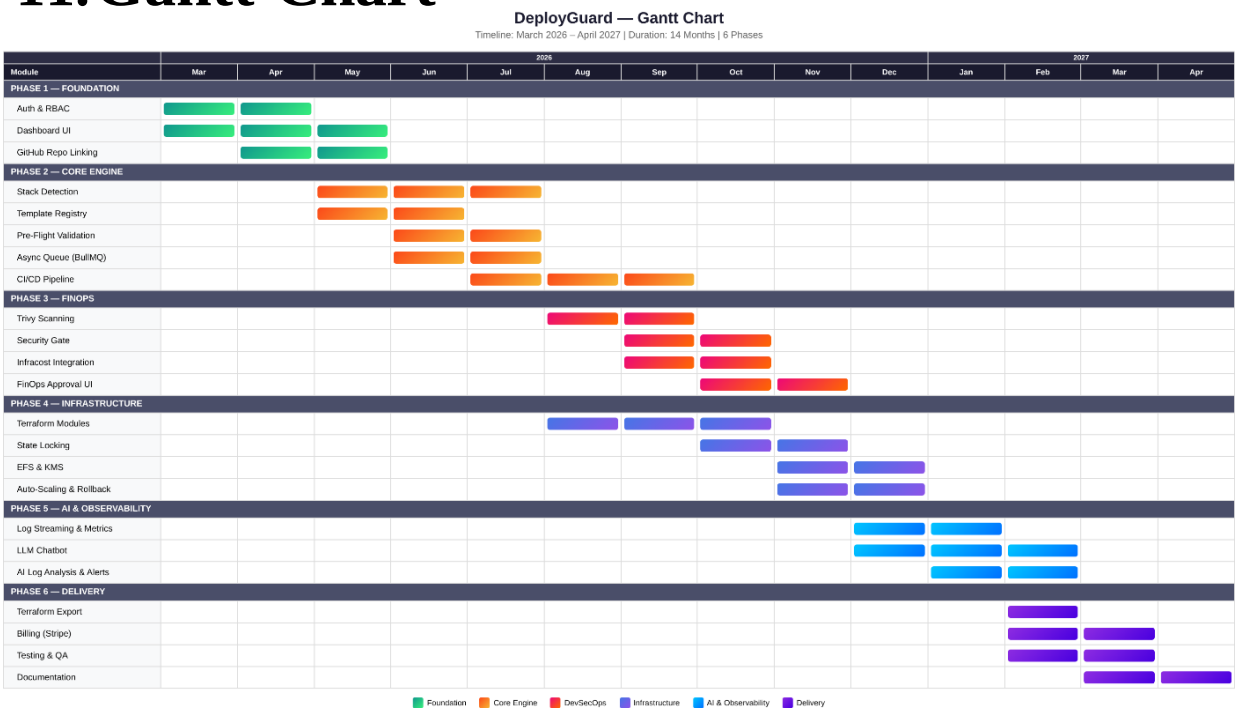
Stakeholder	Role
Air University Islamabad	Project Sponsor
Sir Yaseen Mushtaq	Project Supervisor
Final Year Project Committee	Evaluation and Quality Assurance of Project
Hassan Sajjad (232460)	Project Lead, Cloud Architect & IaC Engineer
Faria Fatima (233169)	Full-Stack Developer & Backend Orchestration Engineer
Tania Khawar (232378)	Full-Stack Developer & AI Integration Specialist

10. Module based Work Division

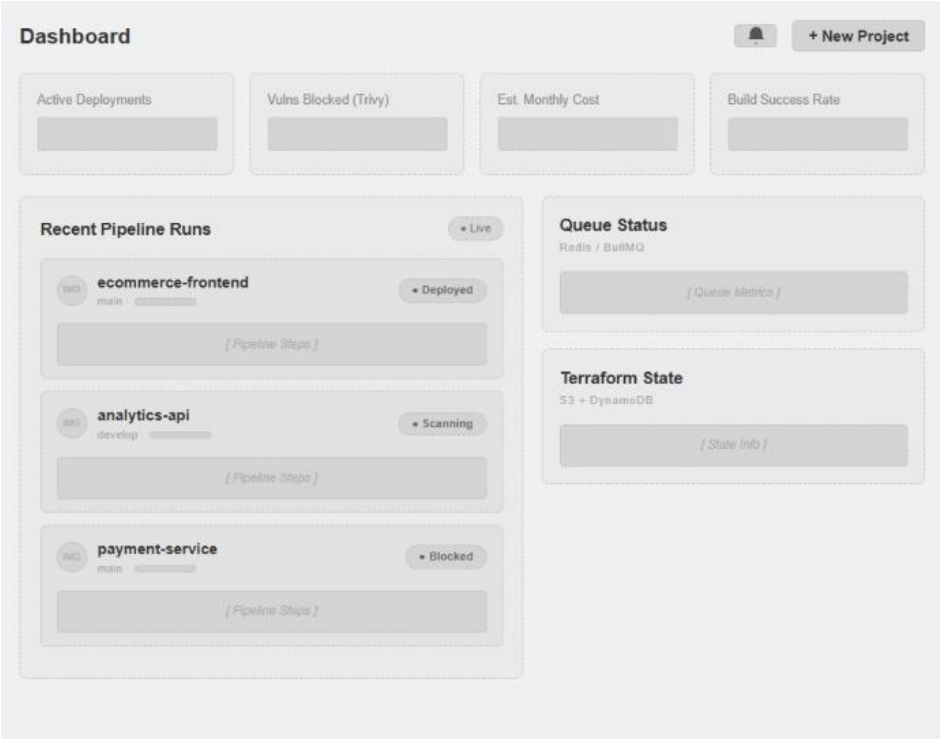
Student Name	Registration	Responsibility / Module / Feature
Hassan Sajjad	**232460**	**- Cloud Architect & Lead –**

		* Infrastructure as Code (Terraform) * Distributed State Locking (S3/DynamoDB) * Security Scanning Gate (Aqua Trivy) * Predictive Cost Analysis (Infracost) * Lifecycle & Environment Demolition
Tania Khawar	**232378**	**- Frontend & AI Specialist -** * Next.js User Dashboard & UI/UX * AI Log Analysis & Chatbot * Log Parsing & Remediation Logic (Optional) * CI/CD Pipeline Visualization * Real-time Monitoring & SSE Streaming
Faria Fatima	**233169**	**- Backend Orchestrator** * NestJS API & User Authentication * Targeted Repository Ingestion (Shallow Clones) * Asynchronous Queuing (Redis/BullMQ) * PostgreSQL Audit Logging * Payment & Subscription Integration

11. Gantt-Chart



12.1. Home/Dashboard Page



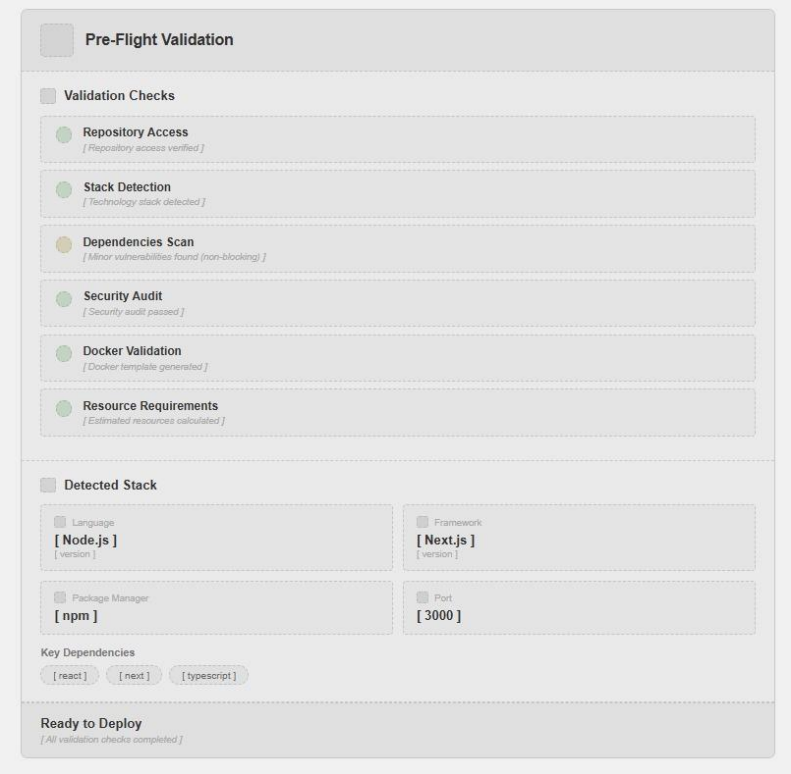
Description: This is the dashboard page, where users can view project summaries, system metrices, current status and recent notifications all in one place.

12.2. Project Creation/Work Space Page



Description: This is the Project Creation page where users link their GitHub repository, select the target branch, and define a project name before proceeding to the next configuration step.

12.3. Repository Stack Detection & Pre-Flight Page



Description: This is the stack detection and pre-flight page, where the system checks the project's technologies, validates the setup, and confirms everything is ready before deployment

12.4. Chatbot Page



Description: This is the AI troubleshooting page where users can analyze deployment logs and chat with a bot to resolve Terraform or build errors.

13. Conclusion

The DeployGuard project addresses a critical infrastructure gap preventing small-to-medium enterprises and startup teams from accessing production-grade cloud deployment capabilities by delivering a comprehensive DevOps orchestration platform built through substantial original engineering work. The platform demonstrates technical depth across multiple computer science domains including rule-based expert systems for security policy enforcement, deterministic stack detection algorithms with hierarchical template matching, custom distributed locking protocols with heartbeat-based failure recovery, intelligent log preprocessing pipelines for AI-assisted troubleshooting, and governed workflow orchestration integrating containerization, infrastructure-as-code provisioning, vulnerability scanning, and cost transparency. DeployGuard is not simple tool integration but rather a sophisticated system comprising approximately 6,500-8,500 lines of custom code implementing decision trees, finite state machines, distributed coordination protocols, pattern matching algorithms, and prompt engineering frameworks. By eliminating the DevOps skill gap through zero-configuration automation while enforcing mandatory security gates and pre-deployment cost approval, the platform enables resource-constrained teams to deploy applications with enterprise-level operational confidence previously accessible only to organizations with dedicated infrastructure specialists. The engineering contribution lies in the orchestration logic that transforms independent technologies—Terraform, Docker, Trivy, Infracost, AWS services, and LLM APIs—into a unified, governed deployment system through custom algorithms for stack-to-template mapping, vulnerability-to-decision conversion, concurrent operation serialization, multi-source log correlation, and deployment-aware prompt construction. We are confident this project demonstrates the technical competence, engineering discipline, algorithmic thinking, distributed systems design, and practical problem-solving skills expected of final year Computer Science students while delivering tangible value to real-world users facing deployment complexity, security risks, cost unpredictability, and troubleshooting friction in modern cloud environments.

14. References

1. Amazon Web Services (AWS) Architecture Center. *Best Practices for Amazon ECS and AWS Fargate*.
2. HashiCorp Documentation. *Terraform State Management and Distributed Locking*.
3. Aqua Security. *Trivy: Comprehensive Security Scanner for Containers and Artifacts*.
4. Infracost Documentation. *Cloud Cost Estimation for Terraform*
5. Storment, J. R., & Fuller, M. (2020). *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management.* O'Reilly Media.